

STUDENT PERFORMANCE ANALYTICS SYSTEM

PROJECT REPORT

A Technical Solution for Student Academic Performance Analysis

Then leave some space and type:

Submitted By:

Name: ___KOENA CHAKRABORTY_____

Roll No.: ___26BCE11432_____

Class/Section: ___CSE CORE_____

Submitted To:

Faculty Name: ___CHAYAN PAUL_____

College Name: ___VELLORE INSTITUTE OF TECHNOLOGY BHOPAL_____

Academic Year: 2026-2027

TABLE OF CONTENTS

Introduction

Real-World Problem Identification

Problem Statement

Objectives

Functional Requirements

Major Functional Modules

Clear Input-Output Structure

Logical Workflow

User-Interactive System

Non-Functional Requirements

System Architecture

Algorithm

Flowchart

Tools and Technologies Used

Implementation Design

Testing and Evaluation

Limitations

Future Scope

Conclusion

1. INTRODUCTION

Student Performance Analytics is a technical solution designed to organize and analyze student academic information. Educational institutions maintain different types of student information, such as marks, subject-wise performance, attendance, and other academic records.

When this information is analyzed manually, it can take considerable time and may make it difficult to identify performance patterns or students who require additional academic attention.

The proposed Student Performance Analytics System provides a structured approach for collecting student information, processing the available data, analyzing academic performance, and presenting meaningful results.

The project applies concepts learned in Python programming and system design, including algorithms, data processing, modular programming, input-output handling, logical workflows, flowcharts, and user interaction.

The purpose of the project is to design a complete technical solution to a practical educational problem while following clearly defined functional and non-functional requirements.

2. REAL-WORLD PROBLEM IDENTIFICATION

Educational institutions generate a large amount of student academic data. This data may include student details, subject marks, attendance information, and overall academic performance.

When this information is maintained and analyzed manually, several difficulties can occur. It may be time-consuming to calculate performance, compare results, organize records, and identify areas where students need additional academic support.

Therefore, a technical solution is required that can organize student information and provide a systematic method for analyzing academic performance.

Identified Real-World Problem

The identified problem is:

How can student academic information be organized and analyzed systematically so that student performance can be understood efficiently and areas requiring attention can be identified?

The proposed solution is a Student Performance Analytics System that organizes input data, processes it using defined logic, analyzes performance, and presents the results in a structured form.

3. PROBLEM STATEMENT

The project aims to design a Student Performance Analytics System for analyzing academic information in a structured and systematic manner.

The system should accept student-related information as input, process the data using defined algorithms and logical workflows, and produce useful performance information as output.

The proposed solution should contain clearly defined functional requirements, three major functional modules, a clear input-output structure, a logical workflow, and user interaction.

It should also satisfy important non-functional requirements such as usability, performance, scalability, reusability, reliability, and maintainability.

The project should further demonstrate system architecture, algorithms, flowcharts, implementation design, documentation, and evaluation.

4. OBJECTIVES

The main objectives of the Student Performance Analytics System are:

- To identify a real-world problem related to student academic performance.
- To design a technical solution for organizing and analyzing student data.
- To define clear functional requirements for the proposed system.
- To divide the system into three important functional modules.
- To establish a clear input-output structure.
- To design a logical workflow for processing student information.
- To provide a user-interactive system.
- To apply algorithms for solving the identified problem.
- To represent the system workflow using flowcharts.
- To design an appropriate system architecture.
- To apply Python programming concepts learned during the course.
- To evaluate the proposed solution against its functional and non-functional requirements.
- To design a system that can be extended and improved in the future.

PAGE 7 — FUNCTIONAL REQUIREMENTS

5. FUNCTIONAL REQUIREMENTS

Functional requirements describe the operations and features that the proposed Student Performance Analytics System is expected to provide.

The system is divided into three major functional modules:

Student Data Input and Management

Student Performance Analysis

Report and Result Presentation

The functional requirements define what the system should do when the user provides student information.

PAGE 8 — MODULE 1

6. MAJOR FUNCTIONAL MODULES

6.1 MODULE 1 — STUDENT DATA INPUT AND MANAGEMENT

The first module is responsible for collecting and organizing student-related information.

Input

The module can accept:

Student ID

Student name

Subject details

Marks obtained

Attendance information

Other relevant academic information

Functions

The module should:

Accept student information.

Organize the entered information.

Validate basic input information.

Store the information in a structured form.

Make the data available for further analysis.

Output

The output of this module is a structured student dataset that can be processed by the performance analysis module.

PAGE 9 — MODULE 2

6.2 MODULE 2 – STUDENT PERFORMANCE ANALYSIS

The second module processes the organized student information and performs academic performance analysis.

Input

The module receives:

Student information

Subject marks

Attendance information

Other available academic data

Functions

The module should:

Read the structured student data.

Process the academic information.

Calculate required performance measures.

Analyze subject-wise performance.

Determine overall performance.

Identify performance patterns.

Identify areas requiring additional attention according to defined criteria.

Output

The output consists of processed performance information and analytical results.

6.3 MODULE 3 – REPORT AND RESULT PRESENTATION

The third module is responsible for presenting the results obtained from the performance analysis.

Input

The module receives the results generated by the performance analysis module.

Functions

The module should:

Display student performance information.

Present calculated results.

Provide performance summaries.

Organize analytical information.

Generate or display a student performance report.

Output

The output is a clear and understandable student performance report that allows the user to view the processed information.

PAGE 11 — INPUT-OUTPUT STRUCTURE

7. CLEAR INPUT-OUTPUT STRUCTURE

The proposed system follows a clear Input → Process → Output structure.

INPUT

The system accepts student-related information such as:

Student ID

Student name

Subject marks

Attendance information

Other academic information

PROCESS

The system:

Validates the input.

Organizes the data.

Processes the academic information.

Performs required calculations.

Analyzes student performance.

Generates analytical results.

OUTPUT

The system provides:

Student performance information

Subject-wise analysis

Overall performance information

Performance summaries

Areas requiring attention

Performance report

Overall Structure

INPUT → DATA PROCESSING → PERFORMANCE ANALYSIS → REPORT / OUTPUT

8. LOGICAL WORKFLOW

The proposed system follows a logical sequence for processing student information.

Workflow Steps

Start the system.

Collect student information.

Validate the entered information.

Organize the student data.

Store the information in a structured format.

Process the student data.

Perform performance calculations.

Analyze academic performance.

Generate analytical results.

Present the results to the user.

Generate or display the performance report.

End the process.

Workflow Representation

START



ENTER STUDENT DATA



VALIDATE DATA

↓

PROCESS DATA

↓

ANALYZE PERFORMANCE

↓

GENERATE RESULTS

↓

DISPLAY REPORT

↓

END

PAGE 13 — USER-INTERACTIVE SYSTEM

9. USER-INTERACTIVE SYSTEM

The proposed system is designed to provide interaction between the user and the application.

The user should be able to:

Enter student information.

Provide academic data.

Start the analysis process.

View processed information.

View student performance results.

Access the generated performance report.

The system should provide clear input instructions and understandable output.

The user interaction should be simple so that a user can operate the system without requiring advanced technical knowledge.

The interaction follows the general structure:

USER → INPUT → SYSTEM PROCESSING → RESULTS → USER

PAGE 14 — NON-FUNCTIONAL REQUIREMENTS

10. NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements describe the quality, efficiency, and characteristics that the proposed system should have.

10.1 Usability

The system should be simple and easy to understand. The input and output should be clearly presented so that users can operate the system with minimal difficulty.

10.2 Performance

The system should process student information efficiently and provide results within a reasonable amount of time.

10.3 Scalability

The system should be capable of being extended to handle additional students, subjects, and academic records as the amount of data increases.

10.4 Reusability

The system should use modular components so that individual functions can be reused or modified in future versions.

10.5 Reliability

The system should provide consistent results when valid and correctly structured data is provided.

10.6 Maintainability

The modular design should make it easier to modify, update, debug, and maintain individual components of the system.

PAGE 15 — SYSTEM ARCHITECTURE

11. SYSTEM ARCHITECTURE

The proposed Student Performance Analytics System follows a modular architecture.

The system is divided into separate components, with each component performing a specific responsibility.

Main Architectural Components

1. User Interface Layer

This layer interacts with the user and accepts input information. It also presents the final results.

2. Data Management Layer

This layer organizes and manages student information in a structured form.

3. Performance Analysis Layer

This layer processes the student data and performs the required academic analysis.

4. Report and Output Layer

This layer presents the analysis results and organizes them into a performance report.

Architectural Flow

USER



USER INTERFACE



DATA MANAGEMENT



PERFORMANCE ANALYSIS



REPORT / OUTPUT



USER

PAGE 16 — ALGORITHM

12. ALGORITHM

Algorithm for Student Performance Analytics

Step 1: Start.

Step 2: Accept student information from the user.

Step 3: Validate the entered information.

Step 4: Organize the student information in a structured form.

Step 5: Store the student data.

Step 6: Read the available student data.

Step 7: Process the academic information.

Step 8: Calculate the required performance measures.

Step 9: Analyze the academic performance.

Step 10: Generate the performance results.

Step 11: Display the performance information and report.

Step 12: Stop.

PAGE 17 — FLOWCHART

13. FLOWCHART

The following flowchart represents the logical workflow of the proposed Student Performance Analytics System.

START



ENTER STUDENT DATA



VALIDATE DATA



IS DATA VALID?

→ NO → SHOW MESSAGE

→ YES → PROCESS DATA



ANALYZE PERFORMANCE



GENERATE RESULTS



DISPLAY REPORT



END

PAGE 18 — TOOLS AND TECHNOLOGIES

14. TOOLS AND TECHNOLOGIES USED

The following tools and technologies are used or proposed for the project:

Programming Language

Python

Python is used as the primary programming language for implementing the proposed system.

Development Environment

Visual Studio Code

Visual Studio Code is used as the development environment for organizing and editing the project files.

Data Storage

CSV / Structured Data File

A structured data file can be used to store student information for processing and analysis.

Documentation

Microsoft Word

Microsoft Word is used for preparing the project report and documentation.

System Design

Algorithms, Flowcharts and Architecture Diagrams

These are used to represent the logical processing and overall system structure.

Programming Concepts

The project applies concepts such as:

Variables

Input and output

Conditional statements

Loops

Functions

Data processing

Modular programming

File handling

Algorithms

Logical problem solving

15. IMPLEMENTATION DESIGN

The proposed Student Performance Analytics System is designed using Python programming concepts and a modular approach.

The implementation is divided into three major functional areas:

15.1 Data Input and Management

This part is responsible for accepting and organizing student information.

15.2 Performance Analysis

This part processes the student information and performs the required calculations and analysis.

15.3 Report and Result Presentation

This part presents the processed information to the user in an understandable form.

The implementation follows the defined input-output structure and logical workflow.

The modular approach makes the system easier to understand, modify, maintain, and extend.

The design applies programming concepts learned during the course, including input/output handling, conditional logic, functions, data processing, algorithms, and modular programming.

PAGE 20 — TESTING AND EVALUATION

16. TESTING AND EVALUATION

Testing and evaluation are important for determining whether the proposed system satisfies its requirements.

The system design can be evaluated using the following criteria:

Test Area	Expected Result
Student data input	Student information should be accepted in the defined structure.
Input validation	Invalid or incomplete information should be identified.
Data processing	Valid information should be processed according to the defined logic.
Performance analysis	Required performance calculations should be performed.
Result generation	Analytical results should be generated from the processed data.
Report presentation	Results should be presented in an understandable format.
User interaction	The user should be able to provide input and view results.
Usability	The workflow should be simple and understandable.
Scalability	The design should allow additional student records and subjects.
Reusability	Modular components should be capable of reuse or modification.

Evaluation Summary

The proposed design addresses the identified real-world problem through clearly defined requirements, three functional modules, a logical workflow, input-output structure, user interaction, modular architecture, algorithm, and flowchart.

The evaluation criteria provide a basis for verifying the system when the implementation is executed and tested.

PAGE 21 – LIMITATIONS

17. LIMITATIONS

The proposed Student Performance Analytics System has the following limitations:

The quality of the analysis depends on the accuracy of the input data.

The current design focuses primarily on academic performance analysis.

Advanced predictive analytics are not included in the basic design.

Integration with institutional databases would require additional development.

Security and authentication features would need to be strengthened before deployment in a real educational institution.

A larger-scale implementation would require additional testing with larger datasets.

PAGE 22 – FUTURE SCOPE

18. FUTURE SCOPE

The Student Performance Analytics System can be enhanced in several ways in the future.

A graphical dashboard can be added for easier visualization of student performance.

Database integration can be implemented for storing larger amounts of student information.

Automated report generation can be added.

Login and authentication features can be introduced.

Detailed attendance analysis can be included.

Performance trends can be analyzed over multiple examinations or semesters.

Notifications can be provided for students who require additional academic attention according to predefined criteria.

The system can be extended to support larger datasets.

A web-based or mobile-based interface can be developed.

Machine learning and predictive analytics can be introduced in future versions.

PAGE 23 – CONCLUSION

19. CONCLUSION

The Student Performance Analytics System provides a structured technical solution to the real-world problem of organizing and analyzing student academic information.

The project identifies the problem and proposes a solution based on clearly defined functional and non-functional requirements.

The system is divided into three major modules: Student Data Input and Management, Student Performance Analysis, and Report and Result Presentation.

The project also demonstrates the use of clear input-output structures, logical workflows, user interaction, modular architecture, algorithms, and flowcharts.

The non-functional requirements of usability, performance, scalability, reusability, reliability, and maintainability have also been considered in the system design.

The project demonstrates how Python programming and system-design concepts learned during the course can be applied to design a technical solution for a practical educational problem.

The proposed design can be further developed and enhanced in the future through database integration, graphical dashboards, automated reporting, web interfaces, and advanced analytics.

Python Modules and Classes

The Student Performance Analytics project is designed using separate Python modules to organize the program into different functional parts. The modules help in maintaining a clear structure and make the project easier to understand and manage.

Module 5 – Student Data Management

This module is responsible for storing and managing student information such as student name, roll number, marks, subjects and other relevant details.

Module 6 – Data Processing

This module processes the student data and performs the required calculations and operations on the collected information.

Module 7 – Performance Analysis

This module analyses the academic performance of students using another performance-related parameters.

Module 8 – Report Generation

This module is responsible for preparing the analysed student performance information in a clear report format.

Module 9 – Data Visualization

Module 10 – Main Program and Integration

This module integrates the different components of the project and provides the main workflow through which the user can interact with the Student Performance Analytics system.

Project Structure

The project consists of Python files/modules for data management, processing, analysis, report generation, visualization and integration. The modules are organized separately to provide a clear and systematic structure.

```
Import pandas as pd
```

```
Import matplotlib.pyplot as plt
```

```
def export_csv(data, filename):
```

```
    data.to_csv(filename, index=False)
```

```
    print("CSV report created successfully!")
```

```
def create_chart(data, filename):
```

```
    if data.empty:
```

```
        print("No data available for chart.")
```

```
        return
```

```
    plt.figure(figsize=(12, 6))
```

```
    plt.bar(
```

```
        data["Name"],
```

```
        data["Percentage"]
```

```
    )
```

```
    plt.xlabel("Student Name")
```

```
    plt.ylabel("Percentage")
```

```
    plt.title("Student Performance Analysis")
```

```
    plt.xticks(rotation=45)
```

```
    plt.tight_layout()
```

```
    plt.savefig(filename)
```

```
    plt.close()
```

```
    print("Performance chart created successfully!")
```

```
def text_report(summary, filename):  
    with open(filename, "w") as file:  
        file.write("STUDENT PERFORMANCE ANALYTICS REPORT\n")  
        file.write("=" * 45 + "\n\n")  
  
        for key, value in summary.items():  
            file.write(f"{key}: {value}\n")  
  
    print("Text report created successfully!")
```

Student Performance Analytics

```
|
├─ main.py
├─ module5.py
├─ module6.py
├─ module7.py
├─ module8.py
├─ module9.py
├─ module10.py
|
├─ data
|   └─ student.csv
|
└─ modules
    ├─ __init__.py
    ├─ module5.py
    ├─ module6.py
    ├─ module7.py
    ├─ module8.py
    ├─ module9.py
    └─ module10.py
```

```
import csv
```

```
FILE_NAME = "students.csv"
```

```
def load_students():
```

```
    students = []
```

```
    with open(FILE_NAME, "r", newline="", encoding="utf-8") as file:
```

```
        reader = csv.DictReader(file)
```

```
        for row in reader:
```

```
            row["Maths"] = float(row["Maths"])
```

```
            row["Science"] = float(row["Science"])
```

```
            row["English"] = float(row["English"])
```

```
            row["Attendance"] = float(row["Attendance"])
```

```
            students.append(row)
```

```
    return students
```

```
def calculate_result(student):
```

```
    total = student["Maths"] + student["Science"] + student["English"]
```

```
    percentage = total / 3
```

```
    if percentage >= 90:
```

```
        grade = "A+"
```

```
    elif percentage >= 80:
```

```
        grade = "A"
```

```
    elif percentage >= 70:
```

```
        grade = "B"
```

```
    elif percentage >= 60:
```

```
        grade = "C"
    elif percentage >= 50:
        grade = "D"
    elif percentage >= 40:
        grade = "E"
    else:
        grade = "F"

    if (
        student["Maths"] >= 40
        and student["Science"] >= 40
        and student["English"] >= 40
    ):
        result = "Pass"
    else:
        result = "Fail"

    return total, percentage, grade, result
```

```
def view_students(students):
    for student in students:
        total, percentage, grade, result = calculate_result(student)

        print(
            student["Student ID"],
            student["Name"],
            "Percentage:", round(percentag, 2),
            "Grade:", grade,
            "Result:", result
        )
```

```
def subject_averages(students):  
    print("\nSubject Averages")  
  
    for subject in ["Maths", "Science", "English"]:  
        total = sum(student[subject] for student in students)  
        average = total / len(students)  
  
        print(subject, ":", round(average, 2))
```

```
def top_students(students):  
    ranked = sorted(  
        students,  
        key=lambda student: calculate_result(student)[1],  
        reverse=True  
    )  
  
    print("\nTop Students")  
  
    for student in ranked[:5]:  
        percentage = calculate_result(student)[1]  
        print(student["Name"], "-", round(percentage, 2), "%")
```

```
def find_student(students):  
    student_id = input("Enter Student ID: ")  
  
    for student in students:  
  
        if student["Student ID"].lower() == student_id.lower():
```

```
total, percentage, grade, result = calculate_result(student)
```

```
print("\nStudent Details")
```

```
print("Name:", student["Name"])
```

```
print("Class:", student["Class"])
```

```
print("Total:", total)
```

```
print("Percentage:", round(percentage, 2))
```

```
print("Grade:", grade)
```

```
print("Result:", result)
```

```
print("Attendance:", student["Attendance"])
```

```
return
```

```
print("Student not found.")
```

```
def main():
```

```
students = load_students()
```

```
while True:
```

```
print("\n===== STUDENT PERFORMANCE ANALYTICS =====")
```

```
print("1. View all students")
```

```
print("2. Subject averages")
```

```
print("3. Top students")
```

```
print("4. Find student")
```

```
print("5. Exit")
```

```
choice = input("Enter your choice: ")
```

```
if choice == "1":
```

```
    view_students(students)

elif choice == "2":
    subject_averages(students)

elif choice == "3":
    top_students(students)

elif choice == "4":
    find_student(students)

elif choice == "5":
    print("Program closed.")
    break

else:
    print("Invalid choice.")

if __name__ == "__main__":
    main()
```

```
import pandas as pd
```

```
FILE_NAME = "data/students.csv"
```

```
def load_data():
```

```
    try:
```

```
        return pd.read_csv(FILE_NAME)
```

```
    except FileNotFoundError:
```

```
        columns = [
```

```
            "Student_ID",
```

```
            "Name",
```

```
            "Class",
```

```
            "Mathematics",
```

```
            "Science",
```

```
            "English",
```

```
            "Attendance"
```

```
        ]
```

```
        return pd.DataFrame(columns=columns)
```

```
def save_data(data):
```

```
    data.to_csv(FILE_NAME, index=False)
```

```
def add_student(student_id, name, student_class,
```

```
                mathematics, science, english, attendance):
```

```
    data = load_data()
```

```
    new_student = {
```

```
        "Student_ID": student_id,
```

```
        "Name": name,
```

```
"Class": student_class,  
"Mathematics": mathematics,  
"Science": science,  
"English": english,  
"Attendance": attendance  
}
```

```
data = pd.concat(  
    [data, pd.DataFrame([new_student])],  
    ignore_index=True  
)
```

```
save_data(data)  
print("Student added successfully!")
```

```
import pandas as pd
```

```
def calculate_results(data):
```

```
    data = data.copy()
```

```
    subjects = ["Maths", "Science", "English"]
```

```
    data["Total"] = data[subjects].sum(axis=1)
```

```
    data["Percentage"] = data["Total"] / 3
```

```
    data["Percentage"] = data["Percentage"].round(2)
```

```
    data["Grade"] = data["Percentage"].apply(assign_grade)
```

```
    data["Result"] = data[subjects].apply( lambda row: "Pass" if all(row >= 40) else "Fail",  
axis=1  
)
```

```
    return data
```

```
def assign_grade(percentage):
```

```
    if percentage >= 90:
```

```
        return "A+"
```

```
    elif percentage >= 80:
```

```
        return "A"
```

```
    elif percentage >= 70:
```

```
        return "B"
```

```
    elif percentage >= 60:
```

```
        return "C"
    elif percentage >= 50:
        return "D"
    elif percentage >= 40:
        return "E"
    else:
        return "F"
```

```
def subject_average(data):
    subjects = ["Maths", "Science", "English"]
    return data[subjects].mean().round(2)
```

```
def top_students(data, n=5):
    data = calculate_results(data)

    return data.sort_values(
        by="Percentage",
        ascending = False
    ).head(n)
```

```
def find_students(data, student_id):
    data = calculate_results(data)

    result = data[
        data["Student ID"].astype(str).str.upper()
        == str(student_id).upper()
    ]

    return result
```

Challenges Faced

During the development of the Student Performance Analysis Project, several challenges were faced. These included organizing student data correctly, maintaining proper input and output formats, validating marks and attendance values, and handling errors in the Python program.

Another challenge was ensuring that the CSV data was arranged correctly so that student ID, student name, attendance and subject marks were stored in the appropriate columns. Debugging the Python program and maintaining proper communication between different modules also required careful attention.

These challenges helped in understanding the importance of proper data organization, validation, modular programming and systematic debugging.

Learning and Key Takeaways

Through this project, I learned how to apply Python programming concepts to develop a practical student performance analysis system.

The major learnings include:

Understanding modular programming and dividing a project into different components.

Working with student records, marks and attendance data.

Using CSV files for storing and managing data.

Applying input validation and error handling.

Designing a logical workflow for a software solution.

Understanding the importance of functional and non-functional requirements.

Creating flowcharts and system design diagrams.

Improving problem-solving and debugging skills.

Understanding how a Python project can be organized for GitHub submission.

References

Python Documentation — Python programming language and standard library:
<https://docs.python.org/3/>

Python CSV Documentation — Working with CSV files in Python:
<https://docs.python.org/3/library/csv.html>

Python unittest Documentation — Python testing framework:
<https://docs.python.org/3/library/unittest.html>

SQLite Documentation — Database concepts and SQLite reference:
<https://www.sqlite.org/docs.html>

Project Organization and Version Control

The project is organized into separate files and modules to make it easy to understand, maintain, and update. The project includes source code files, data files such as CSV files, scripts, configuration-related files, and other project documentation. The different modules are separated according to their functions, which provides a clear and modular structure.

Git and GitHub are used for version control and project management. They help in maintaining different versions of the project files, tracking changes, and keeping the project source code and related files organized.

Validation and Error Handling

The project includes validation of input data to ensure that the information entered or loaded from the data file is in the expected format. Invalid, missing, or inappropriate input data should be identified before processing.

Error handling is used to prevent unexpected inputs or file-related problems from causing the entire program to stop. Appropriate error messages can be provided to help the user understand and correct the problem. This improves the reliability and usability of the Student Performance Management System.